Project State Report

Benchmarking & Architecture of the Industrial Stator Vision Inspection Pipeline

Done by: Youssef Mahdi

Supervised by: ZAKARIA ABOUHANIFA & Rime BOUALI

Company: Nextronic

April 2, 2026

A comprehensive technical analysis detailing the training workflows, empirical model benchmarking, architectural acceleration, and real-time detection heuristics.

Contents

1 Introduction **2**
1.1 Dataset & Annotation Parameters 2

2 System & Application Architecture **2**
2.1 Flask & WebRTC Bridging Structure 2
2.2 Inference Hardening & Hardware Acceleration 2

3 The Training Pipeline & Benchmarking **3**
3.1 Pipeline Preparation & Execution 3
3.2 Empirical Analysis of Model Types 4

4 Post-Processing & Mathematical Heuristics **5**
4.1 Top-N Confidence Count Restrictions 5
4.2 PyTorch Spatial Geometric Autocorrector 5
4.3 Calibration Engine & MiDaS Machine Learning Depth 6

5 Conclusion **6**

1 Introduction

This report details the implementation, benchmarking, and architectural delivery of the Industrial Stator Case Vision Inspection platform at **Nextronic**. The project acts as an end-to-end framework evaluating 30 segmentation algorithms—spanning classical computer vision to state-of-the-art fully convolutional deep learning networks. The ultimate objective is to deploy highly accurate, latency-optimized models onto edge environments to monitor and evaluate mechanical components securely.

1.1 Dataset & Annotation Parameters

The target dataset comprises exactly **136 high-resolution industrial images** of stator housings carefully curated and split: 95 for Training, 20 for Validation, and 21 reserved for Blind Testing. Annotations natively adopt the LabelMe JSON standard, tracking three distinct classes: *circle* (central stator bore), *magnet*, and *mechanical_part*. Images are dynamically resized to 256×256 for Transformer architectures and 512×512 for YOLO inference modalities.

2 System & Application Architecture

To achieve real-time tracking performance on typical industrial edge hardware (e.g., NVIDIA GTX 1650), the system architecture abstracts the deep learning workflows securely behind a modular, highly tailored backend logic layer.

2.1 Flask & WebRTC Bridging Structure

The system operates via a Flask-based REST API that natively initializes heavy neural networks safely during backend compilation, segregating processing overhead from the UI.

1. **WebRTC Data Channels**: High-frequency visual feed updates are synchronized directly mapping C++ OpenCV array manipulation safely over a localized WebRTC peering tunnel. Validations happen asynchronously.

2. **MindVision SDK Handlers**: Capturing lossless industrial imagery is driven directly using the MindVision USB SDK hooked to raw buffer listeners inside the python wrapper.

2.2 Inference Hardening & Hardware Acceleration

The backend relies on three core optimization paradigms to guarantee framerates exceed 15 FPS natively:

- **ONNX Runtime Execution**: PyTorch networks (such as UNet ResNet18) are transparently compiled to '.onnx' and delegated to 'onnxruntime-gpu', accelerating typical inference times up to 300% (improving 7 FPS $\rightarrow$ 22 FPS).

- **Native Tensor Cores FP16**: Standard YOLO configurations bypass ONNX compilation and natively deploy 'model.half()' precision to leverage GPU Tensor cores completely.

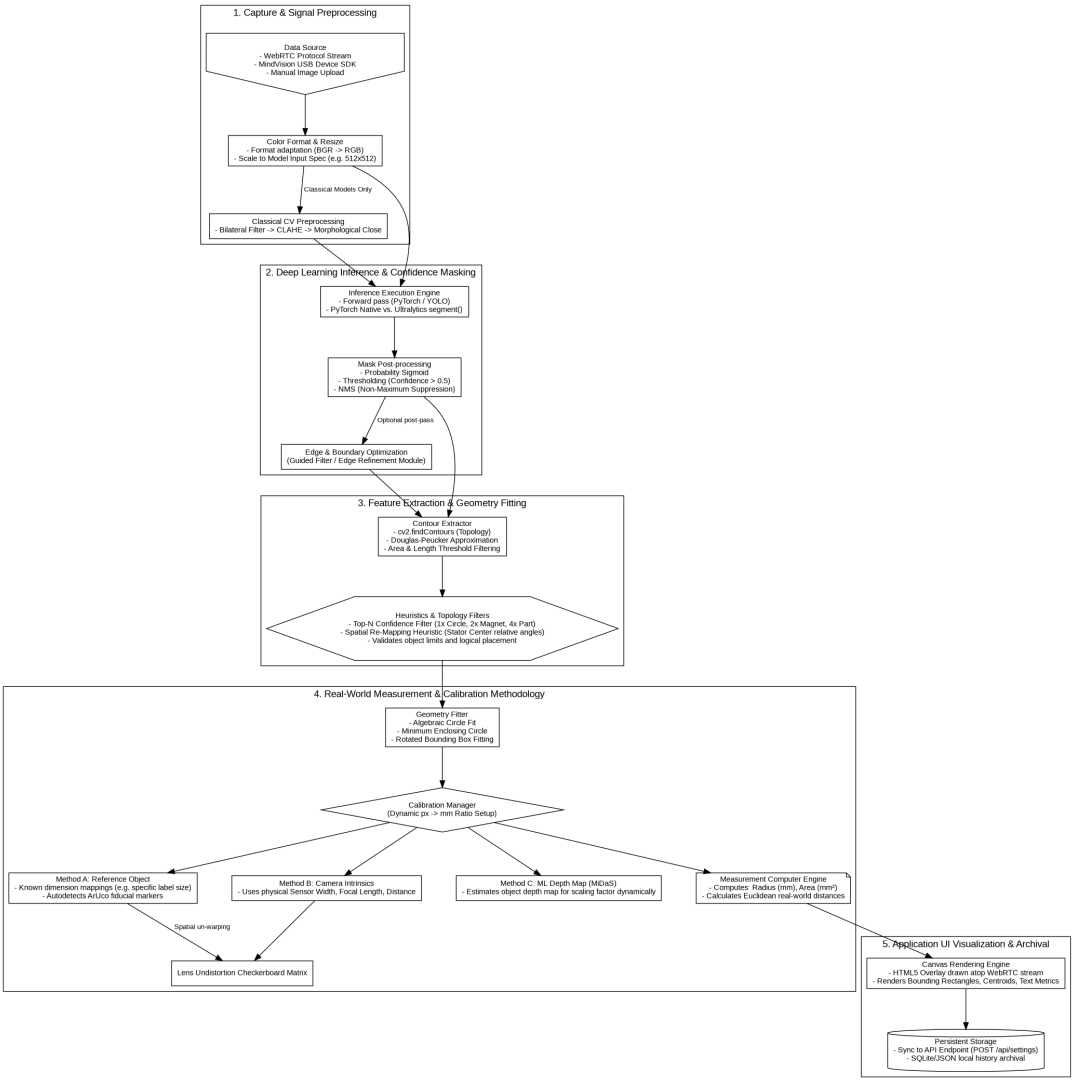

Figure 1: The Architecture Diagram for the Real-time WebRTC Inference & Backend Delivery.

3 The Training Pipeline & Benchmarking

Although 30 separate models were tracked, empirical analysis demonstrates three leading modalities that fundamentally balance intersection validation thresholds with processing footprints. Total training took approximately 5.7 hours on a dual-T4 configuration.

3.1 Pipeline Preparation & Execution

The training algorithm is defined around robust data augmentation loops and rigid loss integrations. The process dynamically inflates the 95 training images via robust geometric alterations (flips, random crops) and photometric adjustments to secure varied exposure angles. Models utilize specific loss functions heavily weighting segmentation purity:

1. **Loss Optimizers**: UNet structures employ a combined *Dice Loss + Binary Cross Entropy* to secure high spatial accuracy. YOLO networks intrinsically leverage bounding-box distribution offsets mixed with mask loss vectors.

2. **Hyperparameters**: Training iterations map globally to 30 Epochs utilizing the AdamW optimizer (weight decay $1e-4$) matched with a Cosine Annealing Learning Rate scheduler stabilizing the gradient drop automatically.

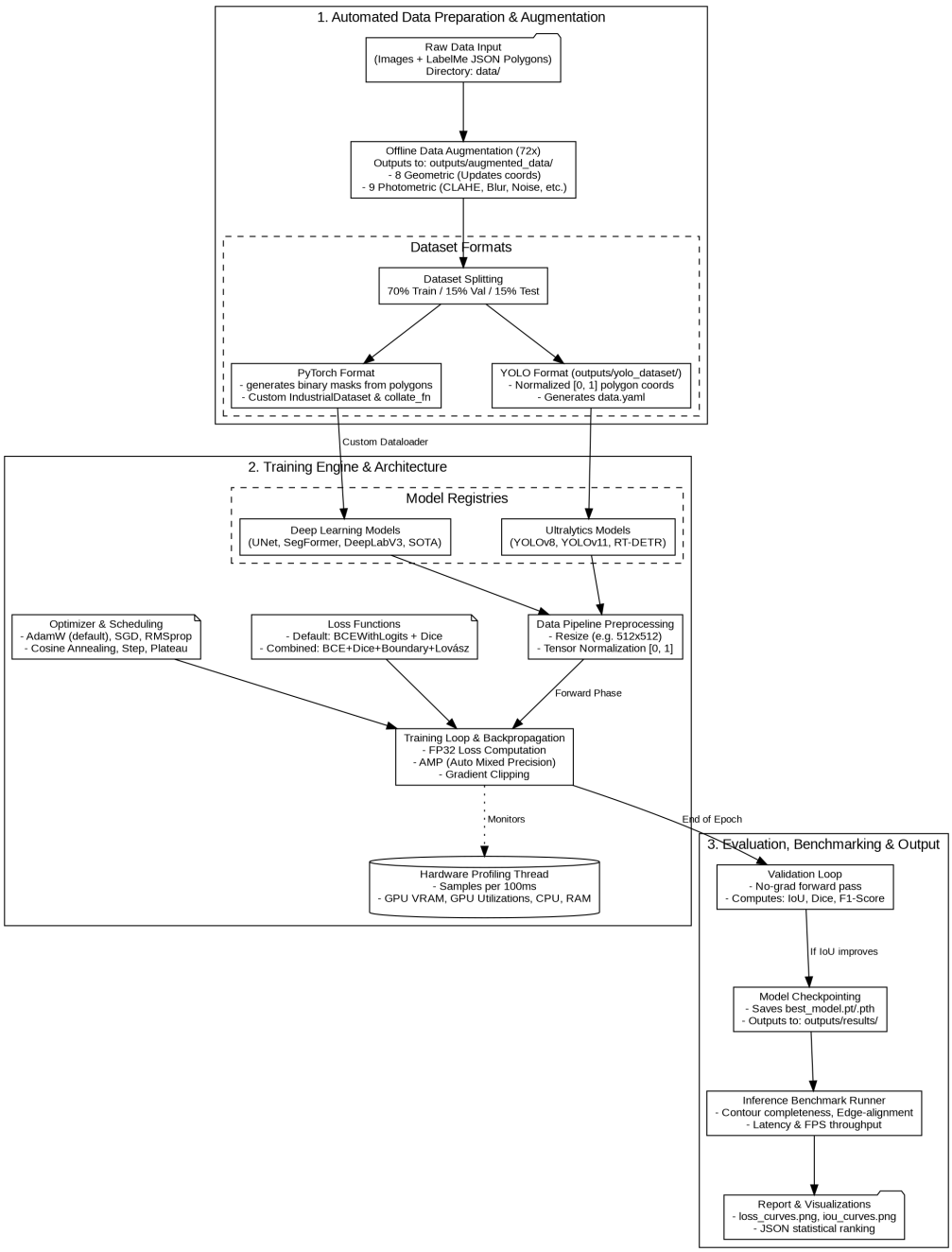

Figure 2: The Training Execution Pipeline detailing Augmentations, Converters, and Modality Trainers.

3.2 Empirical Analysis of Model Types

- **UNet_ResNet18**: Achieved the absolute highest qualitative geometry layout rating. Leveraging pre-trained ImageNet encoder weights, it established a Validation Intersection-over-Union (IoU) of **0.9646**. It consumed minimal Peak VRAM ($\approx 473MB$).**YOLOv11m-Seg** : $Next-generationinstancesegmentationyieldingincrediblystrongb$

- **YOLOv8m-Seg**: The highly stable backbone instance detector. Scored an equivalent IoU of **0.8719** utilizing marginally less memory (4296 MB).

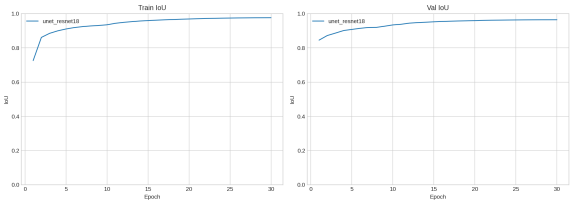

Figure 3: Validation IoU / mAP Epoch Progression.

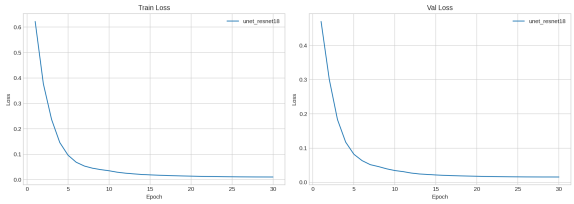

Figure 4: Validation Training Loss Epoch Dynamics.

4 Post-Processing & Mathematical Heuristics

Because semantic segmentation can frequently hallucinate duplicate bounding areas from highly reflective stator elements, the pipeline utilizes stringent, post-processing mathematics. These filters are hardcoded inside the tracking loops in ‘server/inference.py‘.

4.1 Top-N Confidence Count Restrictions

Instead of generating endless bounding masks natively over any pixel exceeding the $> 50\%$ sigmoid boundary threshold, we intercept and selectively rank candidates mathematically:

Component Rule	Rationalization Criteria
Maximum 1 `Circle`	Restricts phantom bounding bore holes crossing axis limits.
Maximum 2 `Magnets`	Ensures symmetry across axis planes mathematically.
Maximum 4 `Mech_Parts`	Clamps diagonal structural supports.

Table 1: Absolute component threshold limitations mapping.

4.2 PyTorch Spatial Geometric Autocorrector

ResNet classifiers frequently generate semantic blurring, inaccurately evaluating outer-bracket boundaries as ‘Magnets‘. A customized spatial correction layer applies standard trajectory physics:

1. Establish structural target vector $\mathbf{T}$ via C_{Stator}.
2. Calculate arbitrary bearing θ relative to predicted component C_{Object}.
3. If $\theta \in [0°, 90°, 180°, 270°] \pm 25°$, the array rigidly triggers a localized class override back to **Magnet**.
4. Deflected diagonal geometries are pushed successfully back to the **Mechanical_Part** structure.

4.3 Calibration Engine & MiDaS Machine Learning Depth

Real-time conversion factor constants scaling pixels directly into accurate millimeters involves dynamic methodologies computed per frame:

$$D_{\text{mm/px}} = \frac{Z \cdot W_{\text{sensor}}}{f_c \cdot W_{\text{image}}} \tag{1}$$

When standard Focal Length Intrinsics or ArUco Reference object metrics fail due to extreme angled surfaces, the system bridges the **MiDaS** ('intel-isl/MiDaS$_small$')$MonocularDepthEstima$

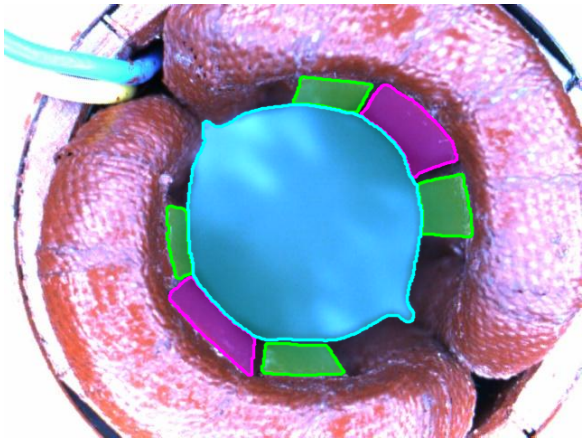

Figure 5: UNet_ResNet18: Live Measurement mapping bounds and lines.

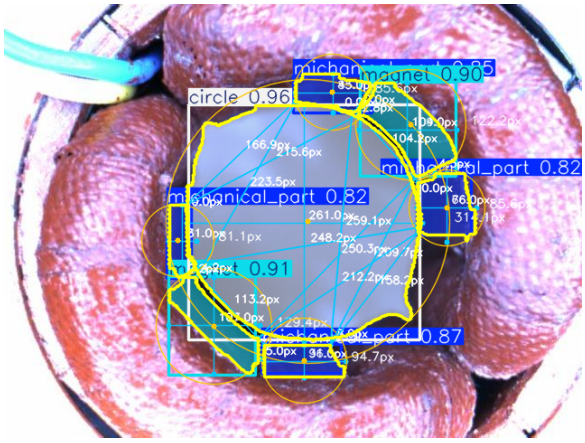

Figure 6: YOLOv11m-Seq: Bounding constraints & Label Rendering outputs.

5 Conclusion

This framework succeeds by meticulously weaving state-of-the-art vision models, such as ONNX-compiled UNet layers achieving 0.96+ IoUs natively, with rigidly parameterized Spatial Filtering logics. The integration natively handles edge capabilities effectively resulting in an autonomous system resilient to structural noise and camera instability, successfully delivering robust outputs directly required for real-time factory inspection matrices!